

Building Libraries: C and C++

Dov Kruger

Department of Electrical and Computer Engineering
Rutgers University

February 9, 2026



Why Libraries?

- Real software is reused across many programs
- Libraries package functionality behind a stable interface
- Separates **interface** (headers) from **implementation** (object code)
- Enables independent compilation, testing, and reuse



Static vs Shared Libraries

- **Static library (.a)**
 - Code copied into executable at link time
 - Larger executables
 - No runtime dependency on the library file
- **Shared library (.so)**
 - Code loaded at runtime
 - Smaller executables
 - One copy of code shared among processes



What Changes When You Use a Library?

- Function calls may cross compilation units
- Inlining may no longer be possible
- Calling conventions and ABI matter
- Performance can change even if the code is identical



Inlining and Function Calls

- **Inline function:** body substituted at call site
- Eliminates call/return overhead
- Enables further optimization (constant folding, vectorization)
- Usually only possible when compiler sees the definition



Why Call Overhead Matters

- Many library functions do very little work
- For small functions, call overhead can dominate runtime
- We will measure the cost of:
 - inline functions
 - static library calls
 - shared library calls



A Concrete Example: Quadratic Roots

- Solve $ax^2 + bx + c = 0$
- Fixed, small amount of floating-point work
- No memory allocation
- Ideal for measuring call overhead



Different APIs for the Same Computation

- Pass outputs by pointer
- Return a small struct
- Return a C++ object
- All compute the same thing
- Performance and ABI behavior may differ



Why Use C at the Library Boundary?

- C ABI is stable across compilers and versions
- No name mangling
- No exceptions, templates, or hidden runtime state
- Most system libraries expose a C interface



C++ and ABI Fragility

- Name mangling depends on compiler
- Object layout may change
- Exception handling crosses library boundaries poorly
- Exporting C++ symbols ties users to your compiler and flags



The Common Pattern

- Implement core functionality with a C interface
- Build static and shared libraries from this code
- Provide C++ wrappers in headers
- C++ wrappers are inline and not part of the ABI



Pass-by-Pointer vs Return-by-Value

- Intuition: returning a struct is expensive
- Reality: small structs are often returned in registers
- Pointer-based APIs force memory stores
- We will measure the difference



What You Will Build

- A tiny math library for quadratic roots
- Inline version in a header
- Static library version
- Shared library version
- Benchmark code to compare them



What You Should Learn

- How libraries are built and linked
- When functions can be inlined
- Why ABI matters
- Why C interfaces are still everywhere
- How design choices affect performance

